

16주차

≡ 다중 선택

개념

목차

1. 8.1 성능 최적화 개요
2. 8.2 하드웨어를 이용한 성능 최적화
3. 8.3 하이퍼파라미터를 이용한 성능 최적화
 - 8.3.1 배치 정규화
 - 8.3.2 드롭아웃
 - 8.3.3 조기 종료

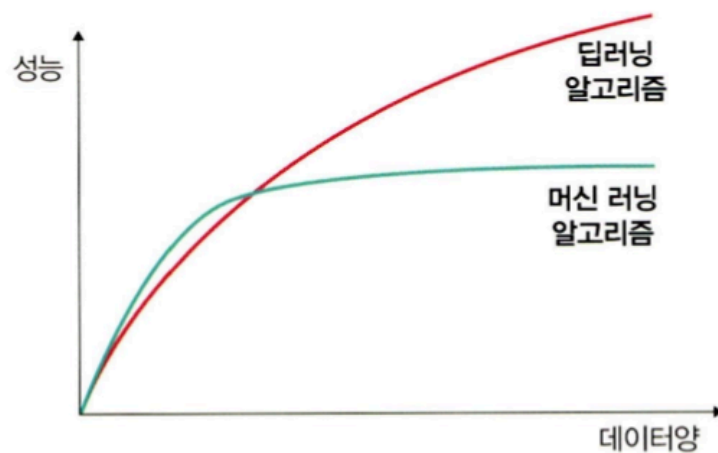
8.1 성능 최적화 개요

딥러닝 성능을 높이는 방법은 크게 데이터 / 알고리즘 / 하드웨어 / 하이퍼파라미터 네 축으로 나뉜다.

8.1.1 데이터를 사용한 성능 최적화

데이터양에 따른 딥러닝 vs 머신러닝 알고리즘 성능 비교 곡선 → 데이터가 많을수록 딥러닝이 머신러닝보다 성능 향상 폭이 크다는 것을 보여주는 그래프.

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- **최대한 많은 데이터 수집:** 딥러닝은 데이터가 많을수록 성능이 올라감.
- **데이터 생성:** 수집이 어려우면 이미지 조작 등으로 인위적으로 생성.
- **데이터 범위(scale) 조정:** 활성화 함수에 맞게 범위를 맞춰야 학습이 안정됨.
 - 시그모이드 사용 시 → 0~1
 - 하이퍼볼릭 탄젠트 사용 시 → -1~1

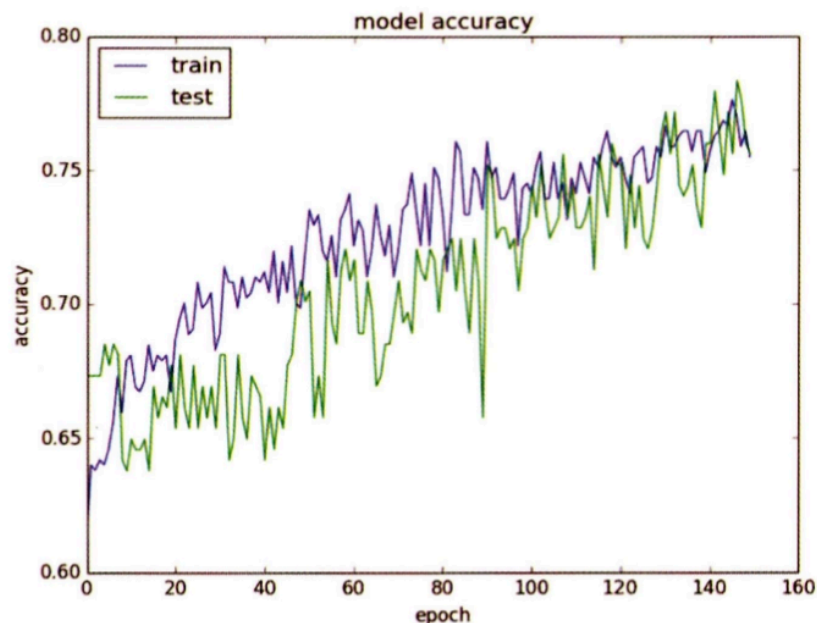
8.1.2 알고리즘을 이용한 성능 최적화

- 수많은 알고리즘 중 최적을 처음부터 알 수 없으므로, 유사한 용도의 알고리즘을 여러 개 훈련시켜 성능이 가장 좋은 것을 선택.
- 분류 → SVM, KNN 등 비교
- 시계열 → RNN, LSTM, GRU 등 비교

8.1.3 알고리즘 튜닝을 이용한 성능 최적화

알고리즘 성능 진단 그래프 (train vs test accuracy, epoch 축) → train이 test보다 눈에 띄게 높으면 과적합, 둘 다 낮으면 과소적합임을 시각적으로 판단하는 기준 그래프.

▼ 그림 8-2 알고리즘 성능 진단



주요 튜닝 항목:

항목	핵심 내용
진단	train/test 곡선으로 과적합·과소적합 여부 파악
가중치	초기값은 작은 난수. 불확실하면 오토인코더로 사전 훈련 후 지도학습 진행
학습률	계층이 많으면 높게, 적으면 낮게. 임의 값에서 시작해 조금씩 조정
활성화 함수	데이터 유형과 목표에 맞게 선택. 출력층은 소프트맥스·시그모이드를 많이 사용
배치 / 에포크	큰 에포크 + 작은 배치가 최근 트렌드. 배치 크기 = 훈련셋 크기와 동일하게 시작해서 테스트
옵티마이저 / 손실함수	SGD가 기본. Adam, RMSProp도 좋은 성능. 다양하게 시도 후 선택
네트워크 구성(토폴로지)	넓게(뉴런 수↑) vs 깊게(층 수↑) vs 혼합. 직접 실험으로 결정

진단 해석 기준:

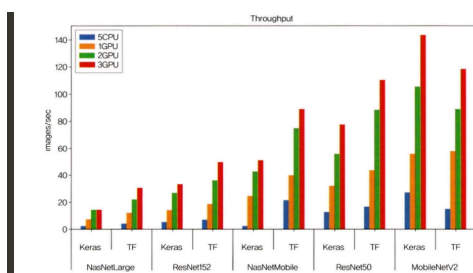
- train >> test → 과적합 → 규제화 적용
- 둘 다 낮음 → 과소적합 → 네트워크 구조 변경 or 에포크 수 증가
- train이 test를 넘어서는 변곡점 → 조기 종료 고려

8.1.4 앙상블을 이용한 성능 최적화

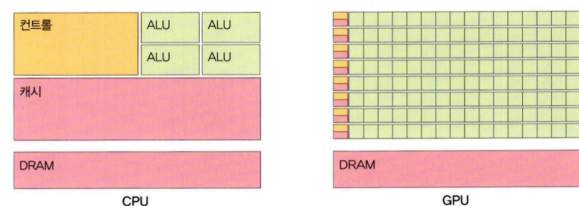
- 모델을 두 개 이상 조합해서 사용하는 방식.
- 하이퍼파라미터 경우의 수를 모두 고려해야 하므로 수십~수백 번 훈련이 필요할 수 있음.

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU 사용의 차이



▼ 그림 8-4 CPU와 GPU의 구조 비교



CPU:

- ALU(연산) + 컨트롤(명령어 해석) + 캐시(데이터 저장)로 구성

- 직렬 처리 방식 — 명령어를 순서대로 하나씩 처리
- 순차 연산(재귀, 행렬의 $A \rightarrow B \rightarrow C$ 열 순서 처리 등)에 적합

GPU:

- 캐시 비중이 낮고 ALU 개수가 매우 많음 (코어 하나에 수백~수천 개)
- 병렬 처리 방식 — 서로 다른 명령어를 동시에 처리
- 역전파처럼 복잡한 미적분 연산을 병렬로 빠르게 처리
- 딥러닝에서 GPU 사용은 선택이 아닌 필수에 가까움

!! 개별 코어 속도는 CPU가 GPU보다 빠르다. 하지만 딥러닝은 수백만 건의 데이터를 벡터 연산으로 처리해야 하므로 병렬 처리에 특화된 GPU가 유리하다.

8.2.2 GPU를 이용한 성능 최적화

CUDA (Computed Unified Device Architecture):

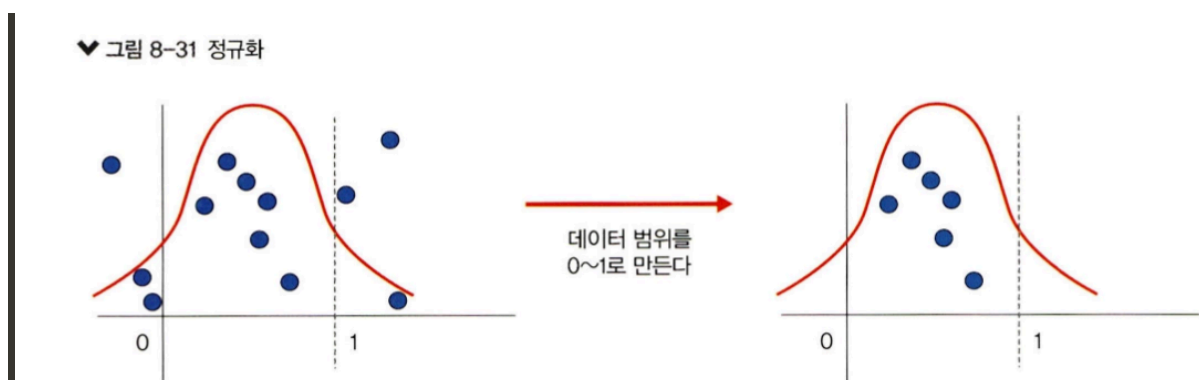
- NVIDIA에서 개발한 GPU 개발 툴
- GPU를 이용한 병렬 프로그래밍을 가능하게 해 줌
- 딥러닝, 채굴 등 대규모 수학적 계산에 많이 사용됨
- CUDA 등장 이후 전문가가 아니어도 GPU 프로그래밍이 가능해짐

8.3 하이퍼파라미터를 이용한 성능 최적화

하이퍼파라미터를 활용한 추가적인 최적화 방법 세 가지: 배치 정규화 / 드롭아웃 / 조기 종료

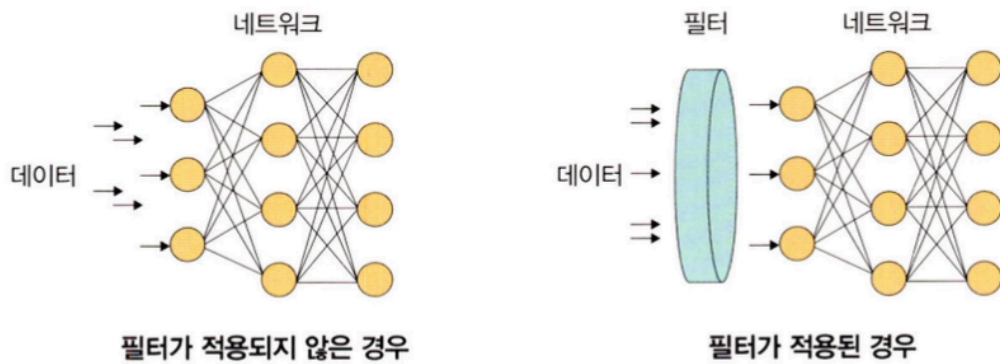
8.3.1 배치 정규화를 이용한 성능 최적화

먼저 용어 구분:



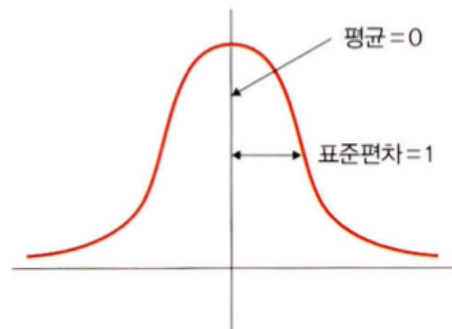
좌: 모든 데이터가 네트워크에 투입. 우: 필터를 거친 데이터만 투입.

♥ 그림 8-32 규제화



표준화 개념 그림 (평균=0, 표준편차=1 정규분포 곡선)

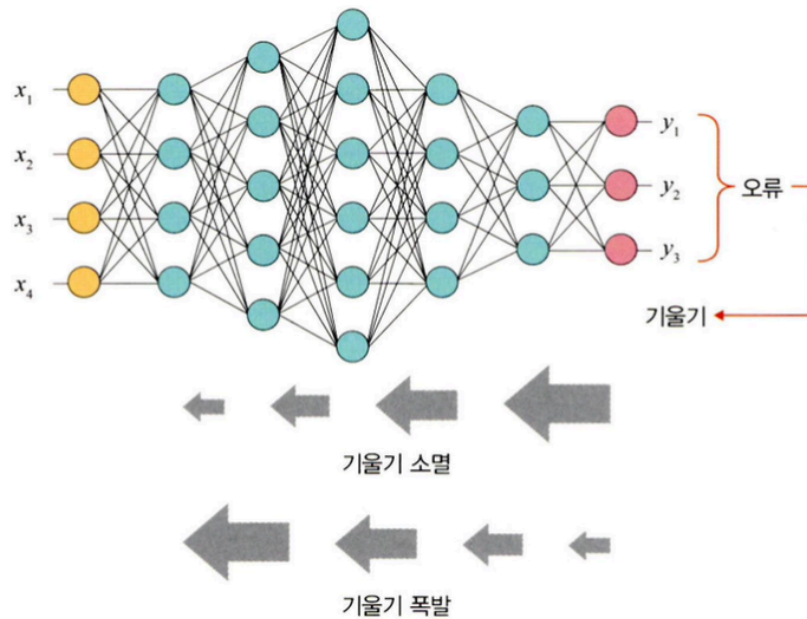
♥ 그림 8-33 표준화



용어	정의
정규화 (Normalization)	데이터 범위를 0~1로 제한. MinMaxScaler 사용. 수식: $(x - x_{\min}) / (x_{\max} - x_{\min})$
규제화 (Regularization)	모델 복잡도를 줄이기 위해 네트워크 입력 전 필터 적용. 드롭아웃·조기종료가 해당
표준화 (Standardization)	평균 0, 표준편차 1인 형태로 변환. z-score 정규화라고도 함. 수식: $(x - m) / \sigma$

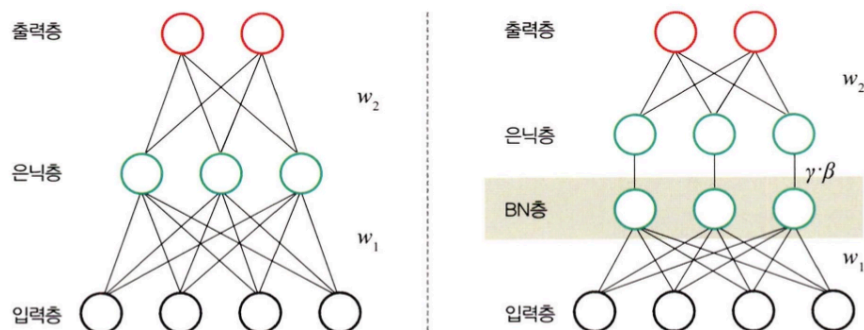
배치 정규화 (Batch Normalization):

역전파 시 기울기가 층을 거슬러 올라가면서 0에 수렴(소멸)하거나 급격히 커지는(폭발) 두 가지 현상



좌: BN 없는 일반 네트워크(입력층-은닉층-출력층). 우: 은닉층과 출력층 사이에 BN층이 삽입된 네트워크.

♥ 그림 8-35 배치 정규화



- 2015년 논문에서 제안. **내부 공변량 변화(Internal Covariate Shift)** 문제를 해결
- 층이 깊을수록 각 층 입력 분포가 계속 바뀌어 학습이 불안정해지는 것을 미니 배치 단위로 정규화해서 방지

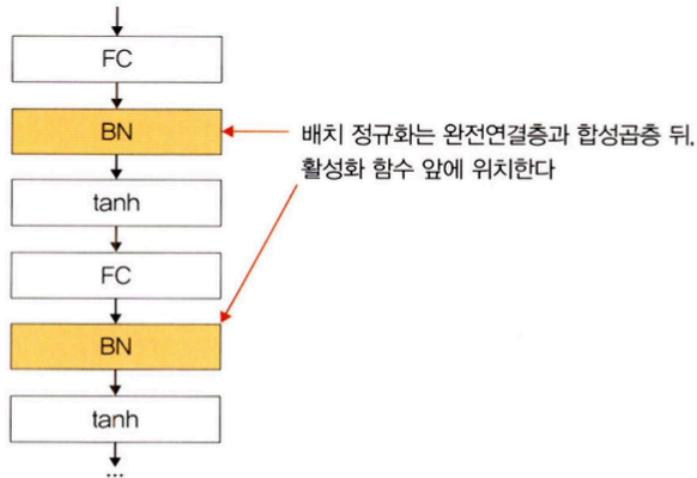
배치 정규화 4단계:

1. 미니 배치 평균 계산
2. 미니 배치 분산·표준편차 계산
3. 정규화 수행

4. 학습 가능한 파라미터 γ (scale), β (shift)로 분포 조정

배치 정규화 삽입 위치: 완전연결층(FC) 또는 합성곱층 뒤, 활성화 함수 앞

▼ 그림 8-40 배치 정규화 위치



장점: 분포가 일정해져 학습 속도 향상. 적용만 해도 대부분 성능이 좋아짐.

단점:

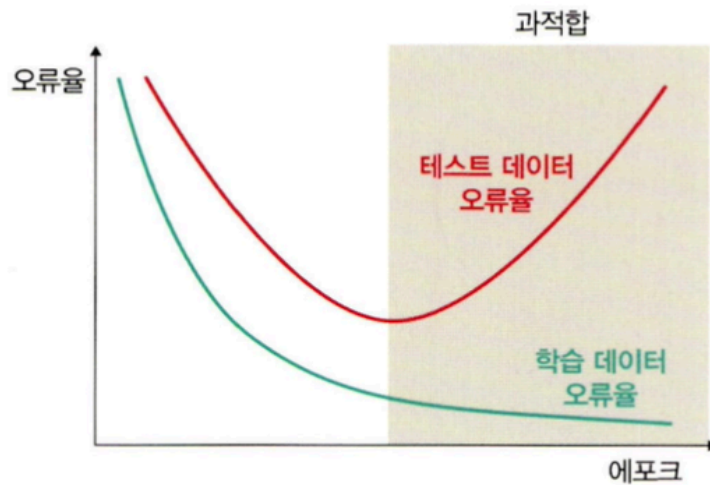
- 배치 크기가 작으면 정규화 값이 불안정 (분산이 0이면 정규화 불가)
- RNN에서는 층마다 따로 적용해야 해서 비효율적

8.3.2 드롭아웃을 이용한 성능 최적화

과적합이란:

에포크가 증가할수록 훈련 오류는 계속 감소하지만 테스트 오류는 어느 순간부터 증가. 두 곡선이 벌어지는 구간이 과적합 영역.

▼ 그림 8-36 훈련과 오류율



- 훈련 데이터셋을 과하게 학습 → 훈련 오류는 낮지만 테스트 오류는 증가하는 상태
- 훈련셋은 실제 데이터의 부분 집합이므로, 새로운 데이터에 일반화가 안 됨

드롭아웃 (Dropout):

좌: 모든 노드가 연결된 일반 신경망. 우: 일부 노드에 X 표시가 되어 비활성화된 드롭아웃 신경망.



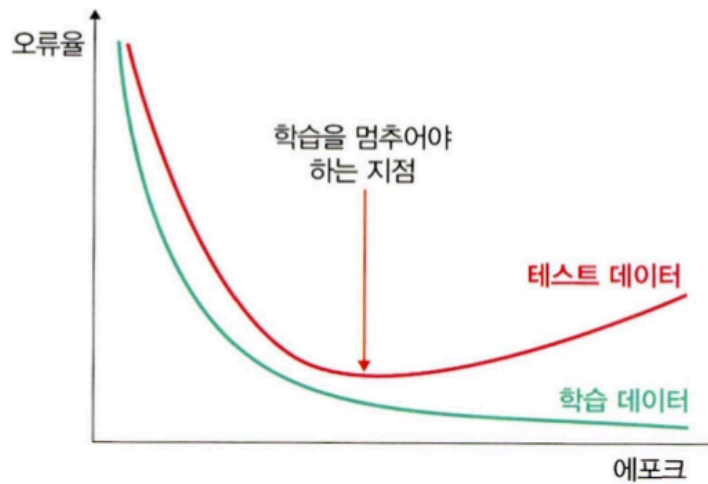
- 훈련 시 일정 비율의 뉴런을 무작위로 끄고 학습
- 꺼진 노드는 신호를 전달하지 않으므로 지나친 학습 방지
- 매 단계마다 사용하지 않는 뉴런을 바꿔가며 훈련 → 다양한 부분 네트워크를 앙상블하는 효과
- 테스트 시에는 모든 노드를 사용하되 드롭아웃 비율을 곱해서 성능 평가
- **단점:** 훈련 시간이 길어짐. 그러나 모델 성능 향상 효과가 커서 널리 사용됨.

8.3.3 조기 종료를 이용한 성능 최적화

조기 종료 (Early Stopping):

에포크 축에서 학습 데이터 오류는 계속 감소, 테스트 데이터 오류는 어느 지점부터 증가. 두 곡선이 교차하는 지점이 학습을 멈추어야 하는 지점.

▼ 그림 8-50 조기 종료



- 매 에포크마다 검증 오차(validation loss)를 측정하여 증가하기 시작하는 시점에서 학습을 멈춤
- 과적합 발생 전까지 훈련 오차와 검증 오차가 함께 감소하다가, 과적합이 발생하면 훈련 오차는 계속 감소하지만 검증 오차는 증가함
- 학습을 언제 멈출지 결정할 뿐, 최고 성능을 보장하지는 않음

핵심 파라미터:

파라미터	설명
patience	개선 없는 에포크를 몇 번까지 기다릴지. 예) 5이면 5번 연속 개선 없으면 종료
delta	개선됐다고 판단하기 위한 최소 변화량. 이보다 적으면 개선 없음으로 처리
verbose	조기 종료 시작/끝을 출력할지 여부

학습률 감소 (Learning Rate Decay)와 함께 사용:

- 학습률을 고정하지 않고 학습 도중 조금씩 낮추는 성능 튜닝 기법
- `ReduceLROnPlateau`: 검증 오차 변동이 없으면 학습률을 factor 배만큼 감소

세 가지 실험 비교 요약:

조건	결과	특징
인수 없음 (기본)	에포크 100 전부 수행	train acc 높음, val acc 변동 심함. 에포크 10 이후부터 val loss 미세 이상향

조건	결과	특징
학습률 감소 적용	에포크 100 전부 수행	완만한 수렴 곡선. val loss 우상향 없음. 에포크 30 수준에서도 좋은 결과 가능
조기 종료 적용	9에포크에서 종료	자원(CPU/메모리) 절약 효과. 성능 항상 보장은 아님

☀ 그래프를 해석해서 어떤 기법이 필요한지 판단하는 것이 중요. 모든 모델에 일괄 적용하는 것이 아니라 상황에 맞게 선택해야 한다!